GRIGORIOS FLOUDAS

MECHANICAL ENGINEERING STUDENT
AT ARISTOTLE UNIVERISTY OF THESSALONIKI

Crane Truss Analysis

Table of Contents

1. Introduction 3

2. Eigenvalue Analysis 5

3. Transient Response Analysis 8

1. Introduction

The present work investigates the structural and dynamic behavior of a small crane, represented as a spatial truss system. The study aims to implement the problem both in HyperMesh and in C++, for numerical computation of static and dynamic responses. The primary objective is to compare the results obtained from the two approaches, including natural frequencies, mode shapes, and nodal displacements, in order to validate the numerical implementation and assess the accuracy and consistency of the computational model. This dual approach provides both a practical demonstration of finite element modeling in commercial software and independent verification through coding.

The crane structure consists of 140 bar elements of identical cross-section and 47 nodes. The fundamental geometry is parameterized with respect to the characteristic length L. The truss was divided into three horizontal levels along the y-axis $(y = 0, y = L/2, y = L)$ and the nodes were numbered sequentially within each level, proceeding from the bottom-left corner upward and from left to right along the $x - z$ plane. Each bar element connects two nodes, and since axial truss behavior is assumed, each node consists of three translational degrees of freedom, resulting in a total of 141 degrees of freedom for the system.

The structural analysis includes:

1. Eigenvalue analysis to determine the first five natural frequencies and corresponding mode shapes.
2. Transient dynamic analysis was performed using a state-space formulation of the equations of motion. The time integration of the state equations was carried out with the Backward Euler implicit method, which enabled the evaluation and comparison of the time-dependent response at a selected node.

The damping matrix was constructed using Caughey's method, assigning a uniform damping ratio of ζ for all modes, ensuring modal damping. The base nodes (1, 2, 31, and 32) were fully restrained to simulate a fixed support condition, and the

corresponding rows and columns were removed from the global mass M, stiffness K, and damping C matrices.

Finally, the computed results from the C++ implementation were compared with the HyperMesh model to verify the accuracy of the numerical solution and the consistency of the dynamic behavior predictions. Figure 1.1 illustrates the geometry of the truss structure as visualized in the HyperMesh software.

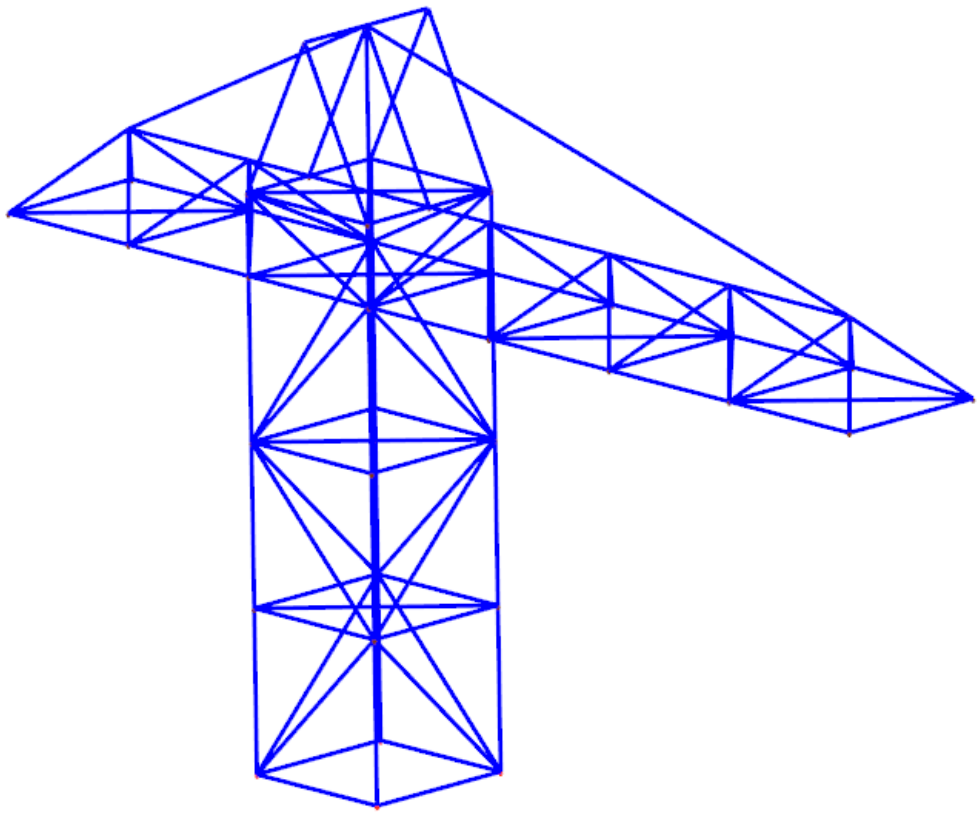

Figure 1.1. *Geometry of the crane truss structure as modeled in HyperMesh.*

2. Eigenvalue Analysis

The dynamic behavior of a linear elastic structure can be described by the second-order matrix differential equation

$$M\ddot{\mathbf{u}}(t) + C\dot{\mathbf{u}}(t) + K\mathbf{u}(t) = \mathbf{f}(t) \tag{2.1}$$

where

M is the global mass matrix,

C is the global damping matrix,

K is the global stiffness matrix,

$\ddot{u}(t)$ is the acceleration vector,

$\dot{u}(t)$ is the velocity vector,

$u(t)$ is the displacement vector, and

$f(t)$ is the external time-dependent force vector.

For free vibration analysis without damping and external forces, the governing equation (2.1) reduces to

$$M\ddot{\mathbf{u}}(t) + K\mathbf{u}(t) = 0 \tag{2.2}$$

By assuming a harmonic solution of the form

$$\mathbf{u}(t) = \boldsymbol{\varphi} e^{i\omega t} \tag{2.3}$$

the generalized eigenvalue problem is obtained:

$$(K - \omega^2 M)\boldsymbol{\varphi} = 0 \tag{2.4}$$

The eigenvalues correspond to the squared natural frequencies ω^2, while the eigenvectors φ represent the corresponding mode shapes.

The mass matrix M is symmetric and positive definite, ensuring that all masses are positive and that kinetic energy is always non-negative. The stiffness matrix K is also

symmetric and positive definite (or positive semi-definite if rigid body modes exist), guaranteeing that the strain energy is non-negative.

In the case of the FEM model, the *Real Eigenvalue Extraction* method was used using the *EIGRL* card in OptiStruct. The first five natural frequencies of the crane model were obtained from the solver output (.out) file and are summarized in Table 2.1.

Mode	Frequency (in Hz)
1	19.68
2	22.07
3	35.55
4	41.32
5	49.87

Table 2.1. *First five natural frequencies of the crane FEM model obtained from OptiStruct.*

For the C++ implementation, the generalized eigenvalue problem

$$\mathrm{K}\boldsymbol{\varphi} = \lambda \mathrm{M}\boldsymbol{\varphi} \tag{2.5}$$

was solved directly using the Eigen C++ library. Specifically, the *GeneralizedSelfAdjointEigenSolver* class was employed, which is designed for problems with symmetric (self-adjoint) stiffness and mass matrices. Since both M and K are positive definite in this problem, the solver ensures real eigenvalues and orthogonal eigenvectors with respect to the mass matrix. Internally, Eigen reduces the generalized problem to a standard symmetric eigenvalue problem by applying a Cholesky factorization of the mass matrix

$$\mathrm{M} = \mathrm{L}\mathrm{L}^{\mathrm{T}} \tag{2.6}$$

where L is a lower-triangular Cholesky factor. By applying this transformation, the problem is reformulated into a standard symmetric eigenvalue problem,

$$\mathrm{C}\mathbf{y} = \lambda \mathbf{y} \tag{2.7}$$

where $\mathrm{C} = \mathrm{L}^{-1}\mathrm{K}\mathrm{L}^{-\mathrm{T}}$. The solver computes the eigenvalues λ, which correspond to the squared natural frequencies $\left(\omega = \sqrt{\lambda}\right)$, and the eigenvectors y. The mode shapes of the original system are then reconstructed by back-substitution,

$$\boldsymbol{\varphi} = \mathrm{L}^{-\mathrm{T}}\mathbf{y} \tag{2.8}$$

It should be noted that, in practice, the matrix C is not explicitly formed. Instead, the transformations involving L^{-1} and $\mathrm{L}^{-\mathrm{T}}$ are carried out implicitly through efficient triangular solves, ensuring both stability and computational efficiency.

The first five natural frequencies obtained from the C++ implementation are presented in Table 2.2 and are compared with those from the FEM solver. Table 2.3 presents a comparison of the first five eigenvalues obtained from the FEM and C++ implementations.

Mode	**Frequency (in Hz)**
1	15.4
2	22.75
3	35.99
4	41.19
5	46.09

Table 2.2. *First five natural frequencies of the crane computed with the C++ implementation.*

Mode	**% Absolute Error**
1	21.75
2	3.08
3	1.24
4	0.32
5	7.58

Table 2.3. *Error of the first five natural frequencies of the crane between FEM and C++.*

3. Transient Response Analysis

Transient response analysis investigates the time-dependent behavior of a dynamic system when it is subjected to external forces or initial disturbances. Unlike static or modal analyses, which provide steady-state or frequency-domain information, transient analysis focuses on how displacements, velocities, and accelerations evolve over time. In the context of the finite element method (FEM), the governing dynamic equation is expressed by (2.1). The goal of transient analysis is to compute the system's time response $u(t)$, which depends not only on the magnitude and frequency content of $F(t)$, but also on the system's natural frequencies and damping characteristics.

In the present implementation, the Backward Euler method, an implicit time integration technique, is adopted for its stability and robustness when dealing with stiff systems. The method discretizes the equations of motion in the time domain, allowing the dynamic equilibrium to be solved incrementally at each time step. This formulation is particularly well-suited for systems with high damping or where accurate long-term stability is desired.

The damping matrix C was constructed using the Caughey damping model in modal form. In this approach, the global damping matrix is expressed in terms of the mass matrix M, the eigenvector matrix Φ, and a diagonal matrix of modal damping coefficients D

$$C = M\Phi D\Phi^T M \tag{3.1}$$

This formulation ensures that the damping matrix remains symmetric and positive definite, while allowing each mode to be assigned its own damping ratio. To efficiently solve this second-order system in the time domain, it was transformed into a first-order state-space representation by defining the state vector

$$\mathbf{x}(t) = \begin{bmatrix} \dot{\mathbf{u}}(t) \\ \mathbf{u}(t) \end{bmatrix} \tag{3.2}$$

The resulting state-space system becomes

$$\dot{\mathbf{x}}(t) = \begin{bmatrix} -M^{-1}C & -M^{-1}K \\ I & 0 \end{bmatrix} \mathbf{x}(t) + \begin{bmatrix} M^{-1}\Lambda \\ 0 \end{bmatrix} \mathbf{u}(t) \tag{3.3}$$

$$\mathbf{y}(t) = \begin{bmatrix} -M^{-1}C & -M^{-1}K \\ I & 0 \\ 0 & I \end{bmatrix} \mathbf{x}(t) + \begin{bmatrix} M^{-1}\Lambda \\ 0 \\ 0 \end{bmatrix} \mathbf{u}(t) \tag{3.4}$$

where:

- A, B, C, and D are the state-space matrices,
- Λ represents the external force application matrix, and
- $u(t)$ is the vector of applied time-dependent loads.

In the implementation, these matrices were assembled using the Eigen library, and the Backward Euler method was used for time integration. This implicit scheme updates the state vector at each time step according to:

$$(I - \Delta t\, A)x_k = x_{k-1} + \Delta t\, Bu_k \tag{3.5}$$

The updated state is then obtained by solving the above linear system:

$$x_k = (I - \Delta t\, A)^{-1}(x_{k-1} + \Delta t\, Bu_k) \tag{3.6}$$

and the output quantities (displacement, velocity, acceleration) are computed from:

$$y_k = Cx_k + Du_k \tag{3.7}$$

The Backward Euler method is unconditionally stable and well-suited for stiff systems, making it an appropriate choice for the dynamic response of the truss structure.

After solving the transient response problem, the displacement results were obtained for all nodes at each time step. To facilitate a direct comparison with the results from OptiStruct, the displacement time history of node 14 (tip of the crane) was extracted and exported to a .csv file. The exported data were then post-processed in MATLAB, where comparative plots between the C++ implementation and the HyperMesh/OptiStruct results were generated.

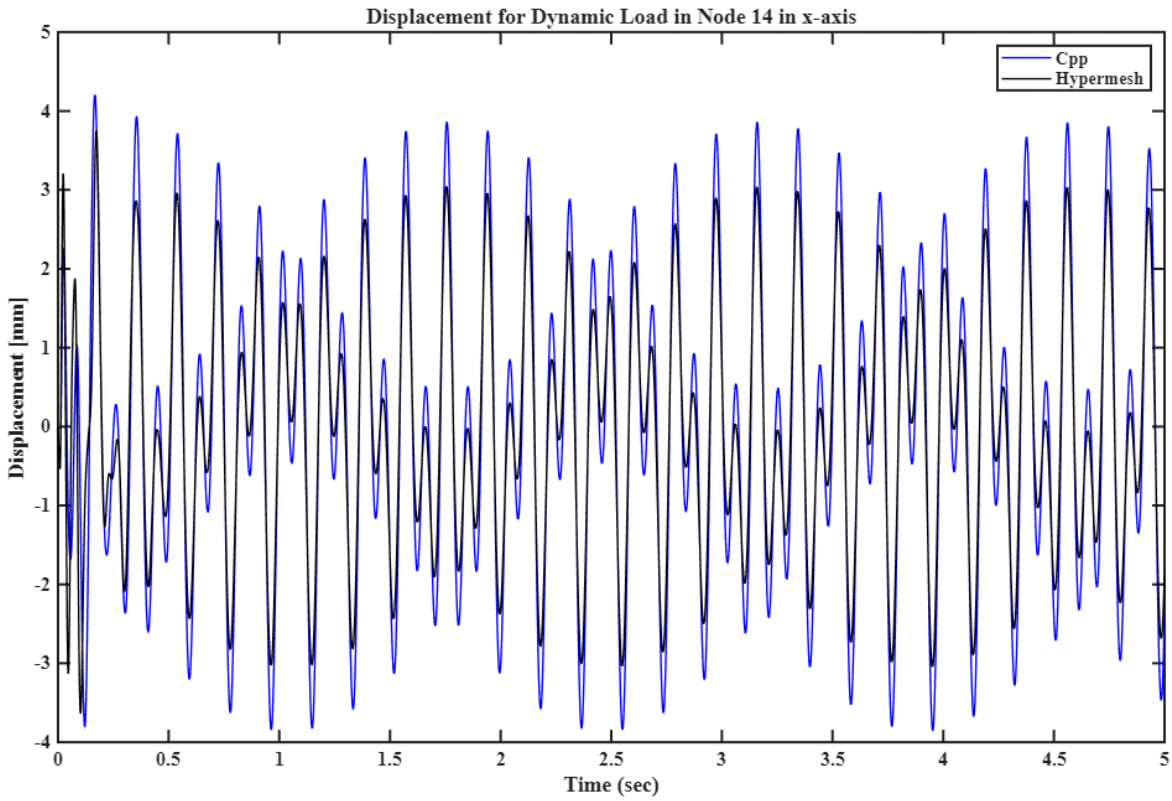

Figure 3.1. *Time history comparison of the displacement in the x-direction at node 14.*

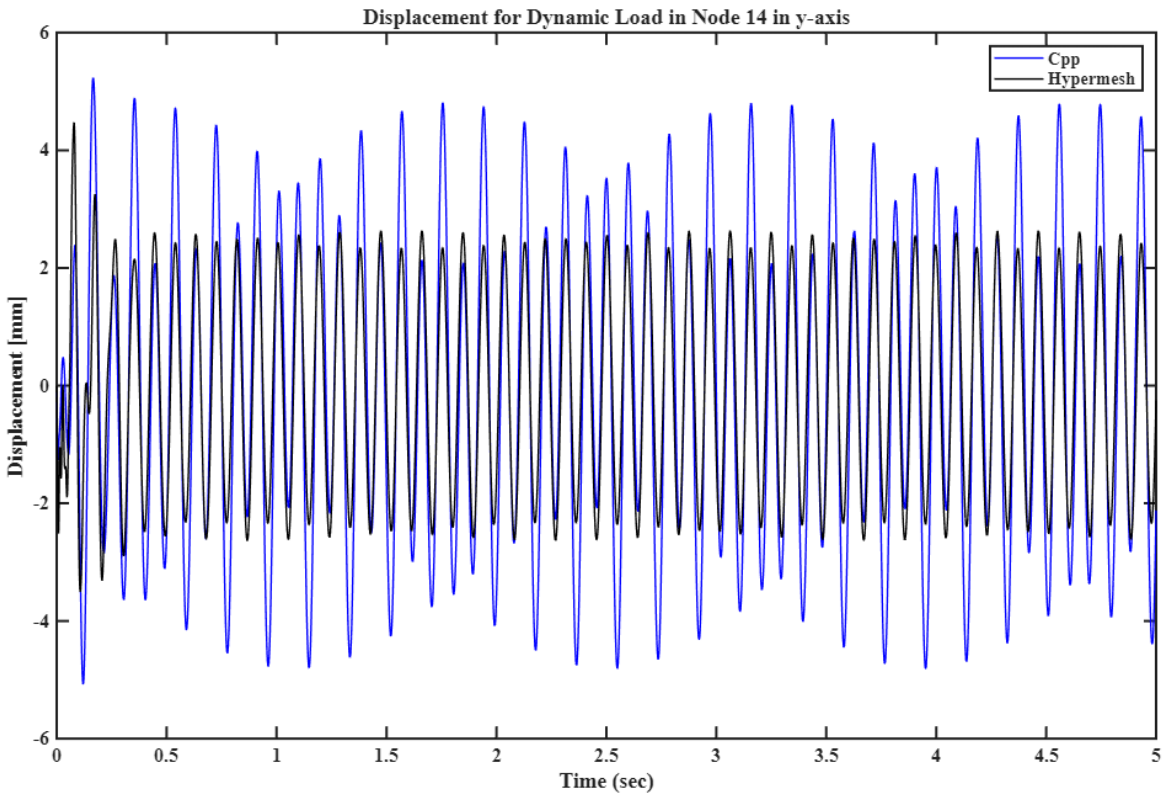

Figure 3.2. *Time history comparison of the displacement in the y-direction at node 14.*

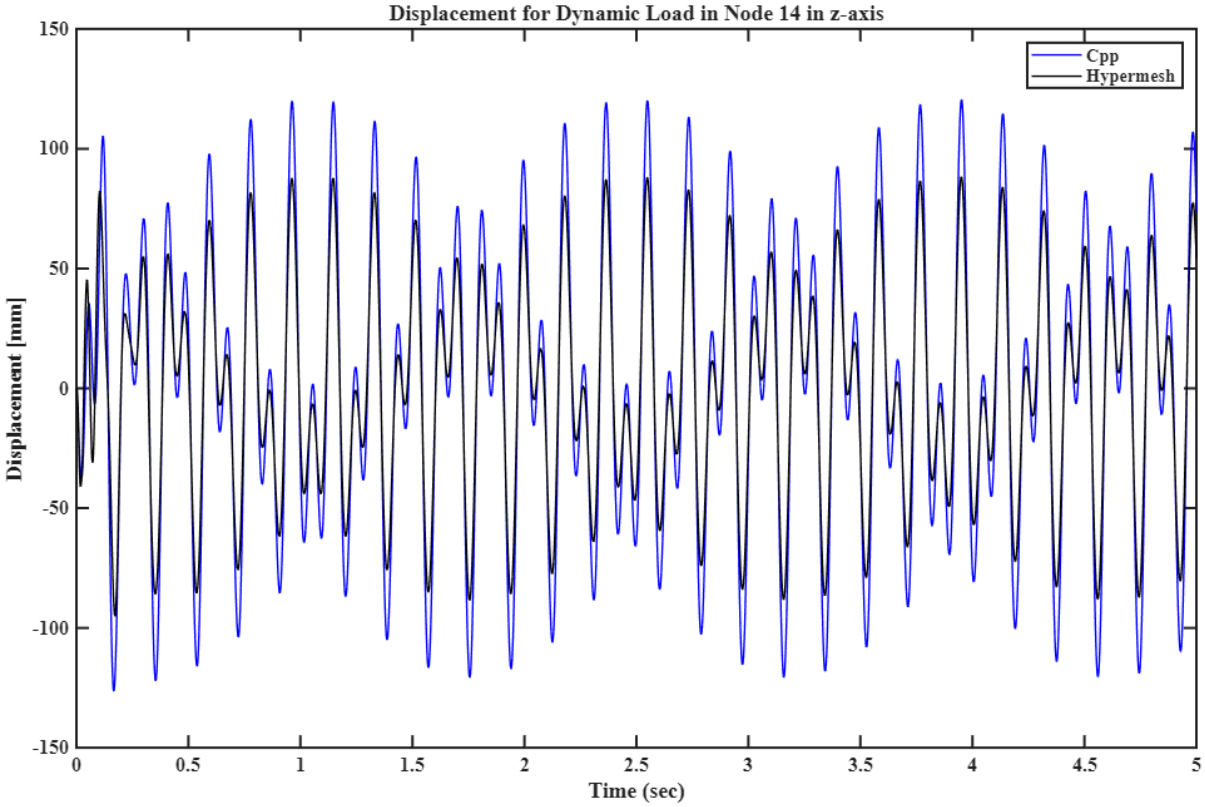

Figure 3.3. *Time history comparison of the displacement in the z-direction at node 14.*